

This is how our code looks like if we don't use Counters. We need to iterate over the entire list and look for each character and update the corresponding count in the defaultdict. We will learn about defaultdict further into this chapter.

```
>>> d = defaultdict(int)
>>> for c in l:
...     d[c] += 1
>>> d
defaultdict(<class 'int'>, {'a':1, 'b':2, 'c':5, 'd':2, 'e':2})
```

This is how we can simplify our code using Counters.

```
>>> Counter(l)
Counter({'c': 5, 'b': 2, 'd': 2, 'e': 2, 'a': 1})
```

defaultdict

Just like Counters, defaultdict too is a subclass of dict. Though most of the functions of dict and defaultdict are pretty similar, there are few stark differences. A defaultdict gives the ability to pass a default_factory in the constructor. The default_factory which accepts a function reference with no arguments, is inherently used to provide default values for non-existent keys.

Let's illustrate this with a basic example

We define a constant function:

```
>>> def constant_factory():
...     return 'missing!'
```

And we define a defaultdict to which we pass our constant function:

```
>>> attendance = defaultdict(constant_factory, {'ram': 'p', 'shayam': 'a', 'rahul': 'p', 'ria': 'p', 'neha': 'a'});
```

The nature of this defaultdict is, whenever a student is found in the attendance, return the corresponding data. If the student is not found, return that the student is missing. The missing data is provided by the constant_factory() function which we passed on to the attendance in its constructor.

```
>>> attendance['ram']
'p'
>> attendance['raj']
```